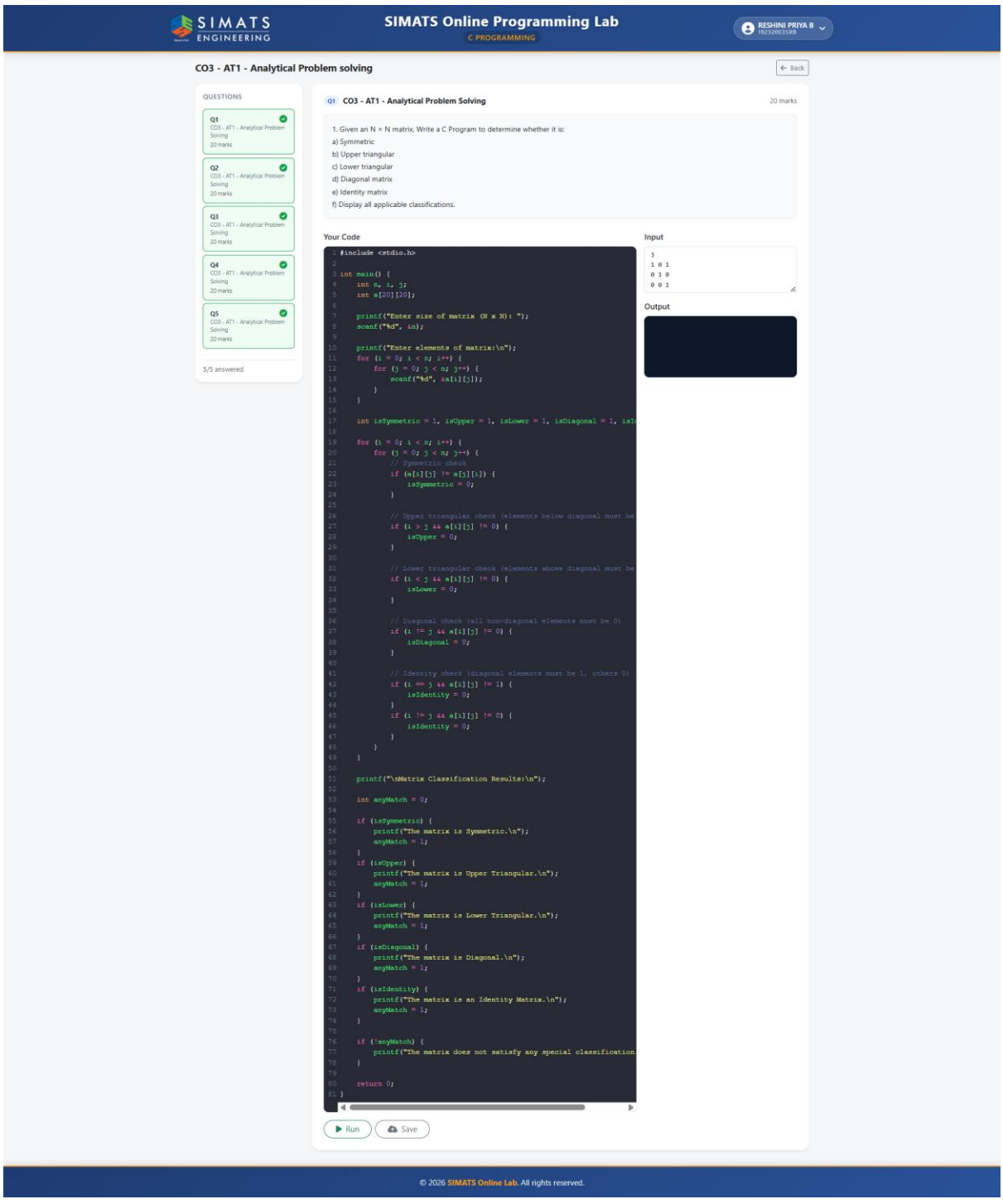
CO3 ASSESSMENT TOOL 1 – Analytical Problem Solving



CO3 - AT1 - Analytical Problem solving

← Back

QUESTIONS

- Q1 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q2 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q3 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q4 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q5 — CO3 - AT1 - Analytical Problem Solving — 20 marks

5/5 answered

Q2 **CO3 - AT1 - Analytical Problem Solving** — 20 marks

2. Given an array of integers, write a C program that:
i. Determines whether the array is sorted.
ii. If sorted, performs Binary Search.
iii. Otherwise, performs Linear Search.
iv. Displays the search method used and the number of comparisons performed.

Your Code

```c
#include <stdio.h>


int isSorted(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        if (arr[i] > arr[i + 1])
            return 0; // Not sorted
    }
    return 1; // Sorted
}

// Linear Search function
int linearSearch(int arr[], int n, int key, int *comparisons) {
    *comparisons = 0;
    for (int i = 0; i < n; i++) {
        (*comparisons)++;
        if (arr[i] == key)
            return i;
    }
    return -1;
}

// Binary Search function
int binarySearch(int arr[], int n, int key, int *comparisons) {
    int low = 0, high = n - 1;
    *comparisons = 0;

    while (low <= high) {
        int mid = (low + high) / 2;
        (*comparisons)++;

        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main() {
    int n, key, comparisons = 0, result;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter the element to search: ");
    scanf("%d", &key);

    if (isSorted(arr, n)) {
        printf("\nArray is sorted.\n");
        printf("Using Binary Search...\n");
        result = binarySearch(arr, n, key, &comparisons);
    } else {
        printf("\nArray is not sorted.\n");
        printf("Using Linear Search...\n");
        result = linearSearch(arr, n, key, &comparisons);
    }

    if (result != -1)
        printf("Element found at index %d.\n", result);
    else
        printf("Element not found.\n");

    printf("Number of comparisons made: %d\n", comparisons);

    return 0;
}
```

Input

```
5
2 3 5 7 4
4
```

Output

```

```

▶ Run Save

CO3 - AT1 - Analytical Problem solving

Back

QUESTIONS

- Q1 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q2 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q3 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q4 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q5 — CO3 - AT1 - Analytical Problem Solving — 20 marks

5/5 answered

Q3 **CO3 - AT1 - Analytical Problem Solving** — 20 marks

3. Store marks of N students in M subjects using a two-dimensional array. Generate a report containing:
a) Student topper
b) Subject topper
c) Class average
d) Subject-wise average
e) Students scoring above the overall average
f) Rank list without modifying the original data

Your Code

```c
#include <stdio.h>

int main() {
    int n, m;
    printf("Enter number of students: ");
    scanf("%d", &n);
    printf("Enter number of subjects: ");
    scanf("%d", &m);

    int marks[n][m];
    int studentTotal[n];
    int subjectTotal[m];
    float rank[n];

    for (int j = 0; j < m; j++)
        subjectTotal[j] = 0;

    // Input marks
    for (int i = 0; i < n; i++) {
        studentTotal[i] = 0;
        printf("\nEnter marks for Student %d in %d subjects:\n", i + 1
        for (int j = 0; j < m; j++) {
            printf("Subject %d: ", j + 1);
            scanf("%d", &marks[i][j]);
            studentTotal[i] += marks[i][j];
            subjectTotal[j] += marks[i][j];
        }
    }

    // a) Student topper
    int topStudent = 0;
    for (int i = 1; i < n; i++)
        if (studentTotal[i] > studentTotal[topStudent])
            topStudent = i;

    // b) Subject topper (highest scorer in each subject)
    int subjectTopper[m];
    for (int j = 0; j < m; j++) {
        subjectTopper[j] = 0;
        for (int i = 1; i < n; i++)
            if (marks[i][j] > marks[subjectTopper[j]][j])
                subjectTopper[j] = i;
    }

    // c) Class average (average of all marks)
    int grandTotal = 0;
    for (int i = 0; i < n; i++)
        grandTotal += studentTotal[i];
    float classAverage = (float) grandTotal / (n * m);

    // d) Subject-wise average
    float subjectAverage[m];
    for (int j = 0; j < m; j++)
        subjectAverage[j] = (float) subjectTotal[j] / n;

    // e) Overall average per student, students scoring above it
    float overallAvgPerStudent = (float) grandTotal / n;

    // f) Rank list without modifying original data
    // Copy studentTotal into a separate array for ranking
    int rankOrder[n];
    for (int i = 0; i < n; i++)
        rankOrder[i] = i;

    // Simple bubble sort on indices based on studentTotal (original
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (studentTotal[rankOrder[j]] < studentTotal[rankOrder[j
                int temp = rankOrder[j];
                rankOrder[j] = rankOrder[j + 1];
                rankOrder[j + 1] = temp;
            }
        }
    }

    // ---------- REPORT ----------
    printf("\n===== STUDENT MARKS REPORT =====\n");

    printf("\na) Student Topper: Student %d with Total = %d\n",
           topStudent + 1, studentTotal[topStudent]);

    printf("\nb) Subject Toppers:\n");
    for (int j = 0; j < m; j++)
        printf("   Subject %d -> Student %d (Marks = %d)\n",
               j + 1, subjectTopper[j] + 1, marks[subjectTopper[j]][j]

    printf("\nc) Class Average (overall): %.2f\n", classAverage);

    printf("\nd) Subject-wise Average:\n");
    for (int j = 0; j < m; j++)
        printf("   Subject %d -> %.2f\n", j + 1, subjectAverage[j]);

    printf("\ne) Students scoring above overall average (%.2f):\n", ov
    for (int i = 0; i < n; i++)
        if (studentTotal[i] > overallAvgPerStudent)
            printf("   Student %d (Total = %d)\n", i + 1, studentTotal

    printf("\nf) Rank List (without modifying original data):\n");
    for (int i = 0; i < n; i++)
        printf("   Rank %d -> Student %d (Total = %d)\n",
               i + 1, rankOrder[i] + 1, studentTotal[rankOrder[i]]);

    return 0;
}
```

Input

```
2
4
5 6
```

Output

Run Save

CO3 - AT1 - Analytical Problem solving

← Back

QUESTIONS

- Q1 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q2 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q3 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q4 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q5 — CO3 - AT1 - Analytical Problem Solving — 20 marks

5/5 answered

Q4 **CO3 - AT1 - Analytical Problem Solving** — 20 marks

4. Given N strings, write a C program to:
a) Sort them lexicographically.
b) Find the longest and shortest strings.
c) Count duplicate strings.
d) Display strings beginning with vowels.

Your Code

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

int main() {
    int n, i, j;

    printf("Enter the number of strings: ");
    if (scanf("%d", &n) != 1) return 1;
    getchar(); // Consume newline

    char strs[n][100];
    char temp[100];

    printf("Enter %d strings:\n", n);
    for (i = 0; i < n; i++) {
        fgets(strs[i], 100, stdin);
        strs[i][strcspn(strs[i], "\n")] = 0; // Remove newline charact
    }

    // a) Sort them lexicographically (Bubble Sort)
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (strcmp(strs[j], strs[j + 1]) > 0) {
                strcpy(temp, strs[j]);
                strcpy(strs[j], strs[j + 1]);
                strcpy(strs[j + 1], temp);
            }
        }
    }

    printf("\nSorted strings:\n");
    for (i = 0; i < n; i++) {
        printf("%s\n", strs[i]);
    }

    // b) Find the longest and shortest strings
    int longestIdx = 0, shortestIdx = 0;
    for (i = 1; i < n; i++) {
        if (strlen(strs[i]) > strlen(strs[longestIdx])) {
            longestIdx = i;
        }
        if (strlen(strs[i]) < strlen(strs[shortestIdx])) {
            shortestIdx = i;
        }
    }

    printf("\nLongest string: %s", strs[longestIdx]);
    printf("\nShortest string: %s\n", strs[shortestIdx]);

    // c) Count duplicate strings
    int duplicateCount = 0;
    // Since the array is sorted, duplicates are adjacent
    for (i = 0; i < n - 1; i++) {
        if (strcmp(strs[i], strs[i + 1]) == 0) {
            duplicateCount++;
            // Skip the rest of the identical strings to count each gr
            while (i < n - 1 && strcmp(strs[i], strs[i + 1]) == 0) {
                i++;
            }
        }
    }
    printf("\nNumber of duplicate string groups: %d\n", duplicateCount

    // d) Display strings beginning with vowels
    printf("\nStrings beginning with vowels:\n");
    int foundVowel = 0;
    for (i = 0; i < n; i++) {
        char first = tolower(strs[i][0]);
        if (first == 'a' || first == 'e' || first == 'i' || first == '
            printf("%s\n", strs[i]);
            foundVowel = 1;
        }
    }
    if (!foundVowel) {
        printf("None\n");
    }

    return 0;
}
```

Input

```
3
5 6 7
4
```

Output

▶ Run Save

CO3 - AT1 - Analytical Problem solving

← Back

QUESTIONS

- Q1 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q2 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q3 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q4 — CO3 - AT1 - Analytical Problem Solving — 20 marks
- Q5 — CO3 - AT1 - Analytical Problem Solving — 20 marks

5/5 answered

Q5 **CO3 - AT1 - Analytical Problem Solving** — 20 marks

5. Accept a string containing repeated characters. Compress it by replacing consecutive repeated characters with the character followed by its count.
Example:
Input:
aaabbbccccdd
Output:
a3b3c4d2
Also calculate the compression ratio.

Your Code

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s[1000];
    char compressed[2000];

    printf("Enter a string: ");
    scanf("%s", s);

    int n = (int)strlen(s);
    int i = 0, k = 0;

    while (i < n) {
        char ch = s[i];
        int count = 1;

        i++;
        while (i < n && s[i] == ch) {
            count++;
            i++;
        }

        compressed[k++] = ch;

        // Add count as digits
        char numStr[20];
        sprintf(numStr, "%d", count);
        int lenNum = (int)strlen(numStr);
        for (int j = 0; j < lenNum; j++) {
            compressed[k++] = numStr[j];
        }
    }

    compressed[k] = '\0';

    int compressedLen = (int)strlen(compressed);
    double ratio = (double)compressedLen / (double)n;

    printf("Compressed Output: %s\n", compressed);
    printf("Original Length: %d\n", n);
    printf("Compressed Length: %d\n", compressedLen);
    printf("Compression Ratio (compressed/original): %.4f\n", ratio);

    return 0;
}
```

▶ Run Save

Input

```
5
6 7 8
```

Output